

Data Engineering Project: Real-Time Ecommerce Data Processing Pipeline

Student: [Abdullah Shamsan]

Date: 15/ 5/ 2026

1. مقدمة المشروع.

بدأ المشروع من نقطة حاسمة وهي توليد بيانات تحاكي الواقع المعقد للتجارة الإلكترونية، حيث لا تأتي البيانات نظيفة دائماً. قمنا باستخدام سكربت بايثون لتوليد تدفق مستمر من المعاملات بصيغة CSV ، مع تعمد حقن "ضوضاء" في البيانات مثل المبالغ السالبة، القيم المفقودة للهوية، والأهم من ذلك تكرار بعض المعاملات (Duplicates) بنسبة تصل إلى 15%. هذا السيناريو فرض علينا بناء مسار في NiFi لا يكتفي بمجرد نقل البيانات، بل يعمل كمصفاة ذكية تضمن جودة وموثوقية البيانات قبل وصولها إلى مستودعات البيانات الضخمة.

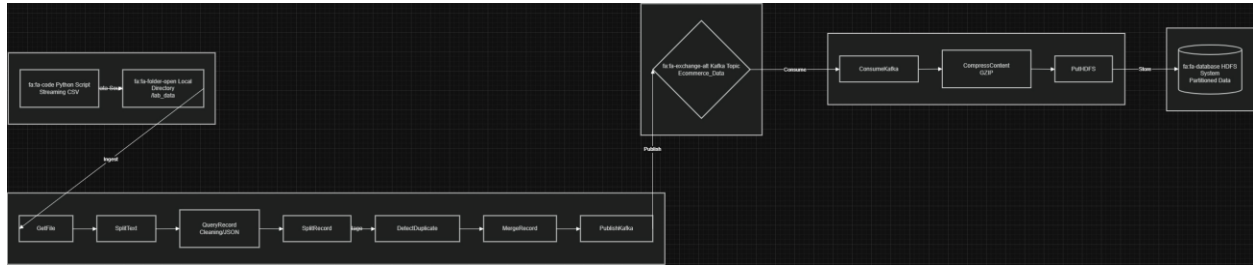
2. المكونات التقنية وبيئة العمل.

- ❖ Apache NiFi أداة إدارة تدفق البيانات
- ❖ Apache Hadoop بيئة التخزين النهائية (HDFS) ضمن حاوية itvdelab-1
- ❖ Docker container بيئة التشغيل
- ❖ Kafka

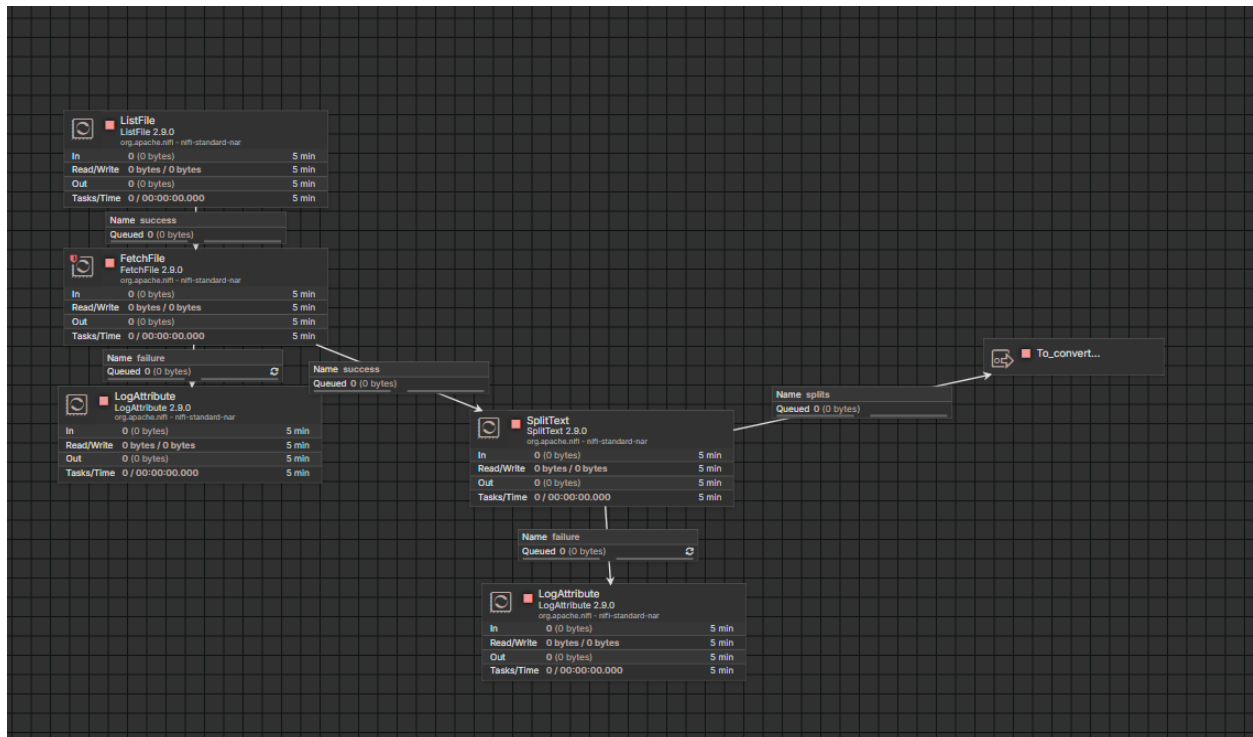
3. Architecture Diagram

يعتمد المخطط الهيكلي لمشروعنا على بناء مسار بيانات متكامل يبدأ من الصفر وينتهي بتخزين احترافي ومنظم، حيث تبدأ الرحلة بسكربت بايثون قمنا ببرمجته ليعمل كمولد بيانات لحظي يحاكي حركة المبيعات في متجر إلكتروني، مع تعمد حقن بيانات "موسخة" ومكررة عشان نختبر كفاءة المسار. هذه البيانات يتم استقبالها في المرحلة الأولى من نيفي عبر معالجات الاستلام التي تقوم فوراً بتقطيع الملفات الكبيرة لحماية الذاكرة، ثم تدخل في مرحلة التنقية العميقة باستخدام لغة SQL لتحويلها لصيغة JSON وتنظيفها من الأخطاء الهيكلية، وبعدها تطبق أهم خطوة وهي فحص التكرار لكل سجل على حدة لضمان إن كل عملية بيع تدخل النظام مرة واحدة بس، ثم ندمج السجلات النظيفة ونرسلها لـ Kafka.

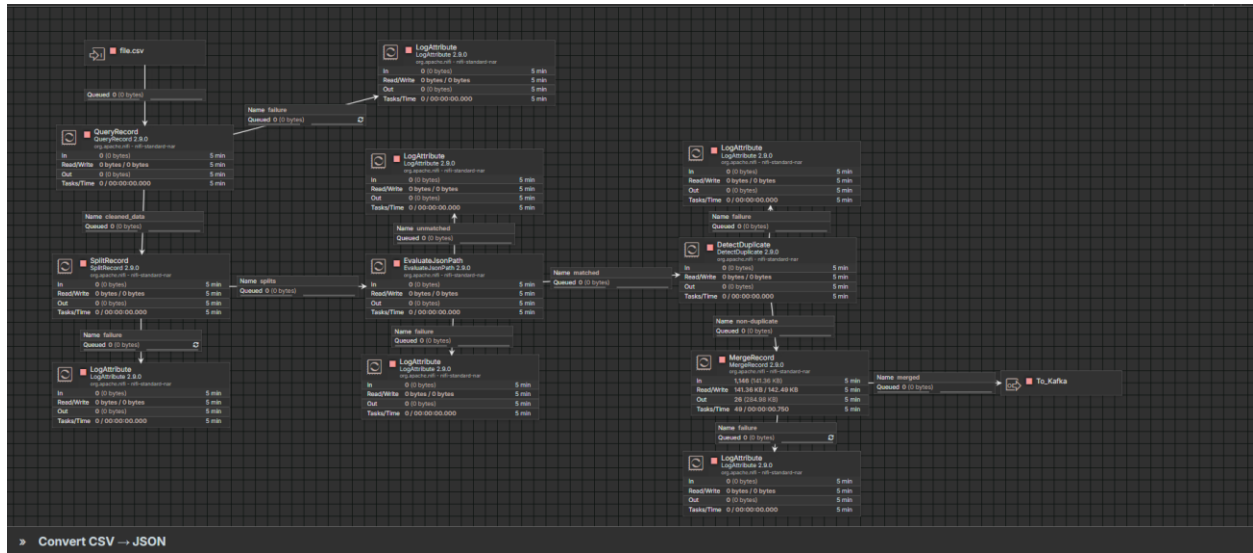
يعمل الكافكا هنا كجسر ذكي يضمن توزيع البيانات وترتيبها بشكل لحظي، حيث يتم توجيه العمليات بناءً على مفاتيح معينة تضمن إن بيانات العميل الواحد تمشي في نفس المسار. وفي النصف الثاني من المسار، يقوم جزء آخر من نيفي بسحب هذه البيانات من الكافكا، وهنا قمنا بإضافة لمسة احترافية وهي عملية ضغط المحتوى بصيغة GZIP ، والهدف منها هو تقليل حجم البيانات للحد الأدنى قبل ما توصل للمحطة الأخيرة في HDFS. هذا الترتيب يضمن لنا إننا نخزن بيانات نظيفة جداً، مرتبة في مجلدات حسب التاريخ، وبأقل مساحة تخزين ممكنة، مع ضمان استقرار النظام وتعامله الذكي مع ضغط البيانات أو أي أخطاء مفاجئة قد تحصل في الشبكة.



4. مرحلة التحليل والتنظيف (Data Transformation & Validation)



في البداية، ملفات الـ CSV التي تطلع من سكربت البايثون تكون أحياناً ثقيلة ومليانة سجلات. لو دخلتها دايركت على معالج الاستعلام (QueryRecord)، كنت خايف يهنق النظام أو تخلص الذاكرة (RAM) لأن الملف كبير. فاستخدمت **SplitText** عشان "أكسر" الملف الكبير لقطع صغيرة وسهلة. كذا نضمن إن نيفي (NiFi) يشتغل وهو مرتاح، ويقدر يعالج كذا قطعة في نفس الوقت (Parallel Processing)، وهذا يسرع الشغل بشكل كبير.



● عملية التنظيف والترتيب (QueryRecord)

هنا بدأت "الفلتر" الحقيقية. الملفات الحين صارت قطع صغيرة، دخلتها على معالج الـ **QueryRecord** استخدمت لغة SQL عشان أقول للنظام: "NiFi"، خذ لي بس السجلات النظيفة، وأي سطر خرابان أو فيه **Corrupted_Row_Error** ارميه ولا تعبره". وفي نفس الوقت، خليته يحول الصيغة من نص (CSV) إلى (JSON) ليش؟ لأن الـ JSON أرتب بمليون مرة، ويخلينا نقدر نتحكم بكل معلومة داخل السجل في الخطوات الجاية بسهولة.

● ليش سويننا SplitRecord مرة ثانية؟

قد يسأل أحد: "ليش قطعت الملفات مرتين؟". الفكرة إن الـ **Split** الأول كان عشان "خفة النظام"، لكن الـ **SplitRecord** الثاني ضروري جداً عشان "منع التكرار". معالج كشف التكرار (**DetectDuplicate**) ما يقدر يفحص ملف فيه 100 سجل ويطلع المكرر منهم؛ هو يحتاج "سجل واحد في كل ملف". فاستخدمت هذا المعالج عشان أفكك المصفوفات وأخلي كل عملية بيع (**Transaction**) تمشي في المسار لحالها، كأنها معاملة مستقلة تماماً.

● كيف صدنا المكرر؟ (EvaluateJsonPath & DetectDuplicate)

بعد ما صار كل سجل لحاله، احتجت طريقة أخلي **nifi** "يشوف" رقم العملية (**Transaction_id**) بدون ما يتعب ويفتح الملف كل شوية. استخدمت **EvaluateJsonPath** عشان يسحب رقم العملية ويحطه كـ "ملصق" (**Attribute**) برا الملف. بعدها يجي دور **DetectDuplicate**، اللي يشيك على هذا الملصق؛ إذا قد مر عليه هذا الرقم من قبل، على طول يسوي له "بلوك" ويرميه في الـ **Duplicate**، وإذا أول مرة يشوفه، يعطيه إذن مرور. كذا ضمنت إن بياناتي نظيفة 100% وما فيها أي عملية مكررة.

● تجميع الملفات في النهاية (MergeRecord)

آخر خطوة كانت "لملة" الشغل. بعد ما فككنا الملفات لسجلات صغيرة عشان نفحصها، ما ينفع نرسلها كذا مبعثرة للـ Kafka أو Hadoop ، لأن كثرة الملفات الصغيرة تعدم الأداء. فاستخدمت **MergeRecord** عشان يجمع السجلات النظيفة اللي نجت من الفحص ويرجع يحطهم في ملف واحد كبير ومنظم. ضبطته إنه يستنى 10 ثواني (Max Bin Age) عشان يلحق يجمع أكبر قدر ممكن من السجلات، وبعدها يكبسهم في ملف واحد ويرسلهم للوجهة النهائية.

❖ التعامل مع السجلات "الخربانة" (Handling Invalid Data)

سكربت البايثون اللي كتبناه كان "خبيث" شوي، لأنه يطلع سجلات مكسورة (Corrupted Rows) وأرقام سالبة غلط. أنا هنا ما خليت النظام يوقف، بل استخدمت **QueryRecord** كفلتر ذكي. أي سجل ما يطابق "الشروط" (Schema) "حقتي، النظام يرميه على مسار الـ **Failure** أو يسوي له استبعاد (Discard) دايركت من خلال استعلام الـ SQL. كذا البيانات اللي وصلت للنهائية هي بس "الزبدة" والنظيفة.

❖ سياسة "توفير الجهد" (Yielding & Resources)

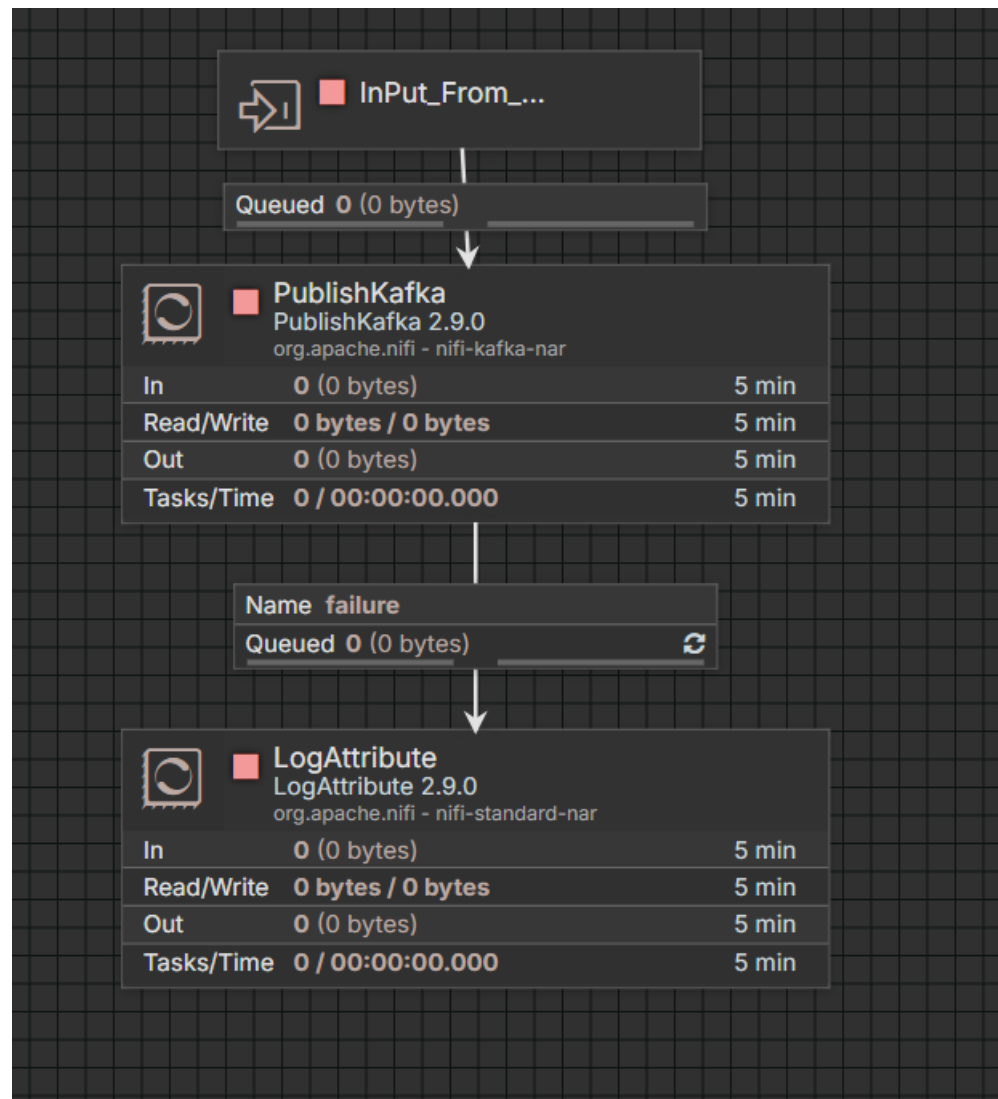
عشان نيفي ما يستهلك المعالج (CPU) حق الجهاز على الفاضي لما ما يكون فيه ملفات جديدة، ضبطت خاصية الـ **Yield** **Duration**. يعني لو المعالج راح يدور ملفات وما لقي شيء، "يرتاح" لمدة 5 ثواني قبل ما يحاول مرة ثانية. هذي الحركة خلت الجهاز بارد ومرتاح، ووفرت الموارد للمعالجات الثقيلة مثل الـ Merge والـ DetectDuplicate.

❖ حل مشكلة "الملفات الصغيرة" (Performance Optimization)

لما فككنا السجلات عشان نفحص التكرار، صار عندنا زحمة ملفات صغيرة جداً. هذي الملفات لو أرسلتها للـ Kafka أو الـ Hadoop بتعدم الأداء وتخلي النظام بطيء جداً. فاستخدمت الـ **MergeRecord** "مكبس"؛ يجمع كل السجلات الصغيرة هذي ويرجع يكبسها في ملف واحد كبير ومحترم قبل ما يطلعها برا نيفي. هذا الشيء حسن سرعة الإرسال (Throughput) بشكل خيالي.

❖ ترتيب الطابور (FIFO Logic)

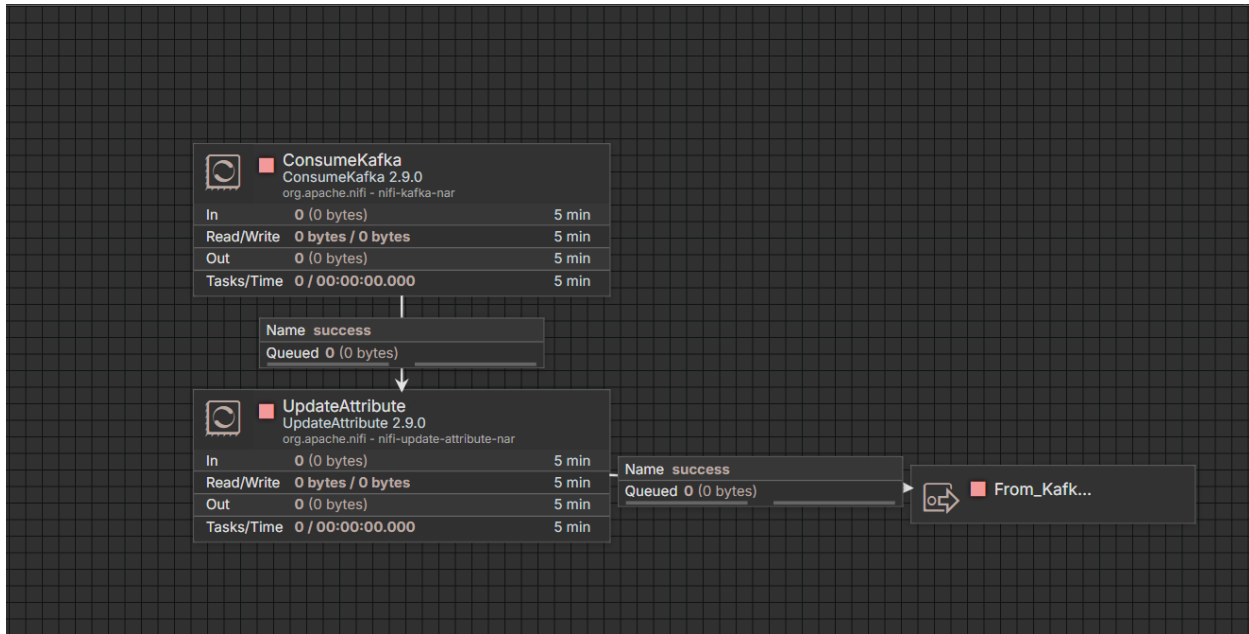
في بيانات التجارة الإلكترونية، التوقيت مهم جداً. فحرصت إن الطوابير كلها تشتغل بنظام **First-In-First-Out (FIFO)**. يعني العملية اللي صارت أول هي اللي تنتظف وتتخزن أول. كذا نضمن إن تسلسل الأحداث (Timestamp) يفضل صحيح وما تتدخل العمليات في بعضها بسبب سرعة المعالجة.



عملية النشر للمواضيع (PublishKafka_2_6)

بعد ما خالصنا تنظيف وتجميع السجلات في ملفات JSON مرتبة، احتجنا نرسلها لـ **Kafka** عشان أي نظام ثاني (مثل Spark أو تطبيق موبايل (يقدر يقرأها لحظياً). استخدمت معالج **PublishKafka**.

- إيش سويت داخل المعالج؟ ضبطت الـ **Topic Name** مثلاً سميته (ecom_orders) هنا نيفي صار يأخذ الملف المدمج ويقطعه لرسائل صغيرة ويرميها داخل الكافكا.
- عملية الـ **Delivery Guarantee** أهم شيء سويته إني ضبطت الـ **Delivery Guarantee** على **Guaranteed Delivery**. هذا يعني إن نيفي ما راح يحذف الملف من عنده إلا لما يتأكد 100% إن الكافكا استلمه وسجله عنده. كذا نضمن إن ولا عملية بيع تضيع في الطريق.



عملية سحب البيانات (ConsumeKafka_2_6)

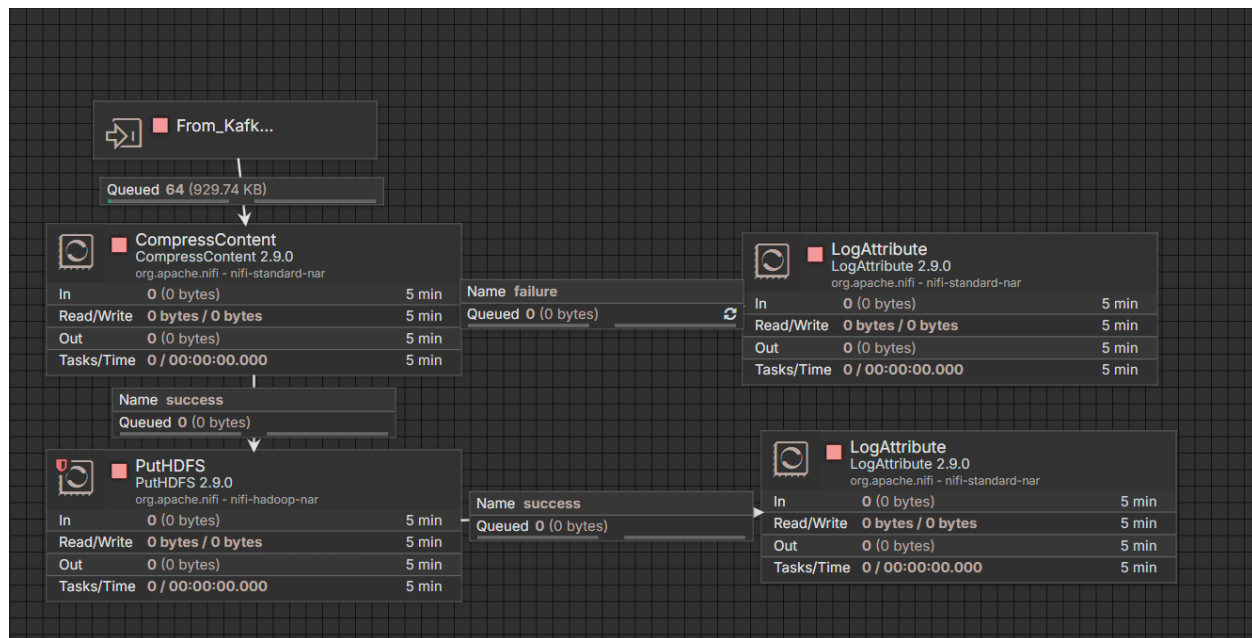
بعد ما البيانات طارت ووصلت للكافكا، احتجنا معالج يسحبها عشان يوديها للمخزن النهائي. استخدمت **ConsumeKafka**.

- إيش اللي صار؟ نيفي هنا شغال كـ "مستهلك" (Consumer). "بمجرد ما يوصل سجل جديد لـ Topic التجارة الإلكترونية، نيفي يسحبه فوراً.
- **ترتيب المجموعة (Group ID):** ضبطت شي اسمه Group ID؛ هذا يخلي نيفي يعرف هو وين وصل في القراءة، يعني لو طفى الجهاز واشتغل، يكمل من السجل اللي وقف عنده وما يعيد القراءة من البداية.

تجهيز المسار في الهادوب (UpdateAttribute for HDFS)

قبل ما أرمي الملف في الهادوب، حبيت يكون الشغل "منظم". استخدمت **UpdateAttribute** عشان أسوي مسار (Path) ديناميكي.

- **الحركة الهندسية:** خليت المسار يعتمد على التاريخ، مثلاً: /ecommerce/data/year=2024/month=05/
- **الفائدة:** هذي الحركة تخلي الـ HDFS مرتب جداً، ولما يجي مهندس البيانات (Data Analyst) يسوي استعلامات بـ Hive أو Spark، تكون البيانات مقسمة (Partitioned) والبحث فيها سريع جداً.



عملية ضغط المحتوى (CompressContent Processing)

بيانات التجارة الإلكترونية التي جالسين نسحبها هي أصلاً نصوص (JSON) ، والنصوص معروف إنها تأخذ مساحة كبيرة وتكرر فيها الكلمات كثير. لو رفعناها للهادوب (HDFS) زي ما هي، بنخلص مساحة التخزين بسرعة والـ Cluster بيتقل علينا. عشان كذا، استخدمت معالج **CompressContent** كخطوة ذكية قبل التخزين النهائي.

عملية الرفع النهائي (PutHDFS)

هنا وصلنا لمحطة القطار الأخيرة. استخدمت معالج **PutHDFS** عشان ننقل الملفات من نيفي إلى قلب الهادوب.

- **الربط :** عطيت المعالج ملفات الإعدادات حق الهادوب (core-site.xml) و (hdfs-site.xml) عشان يقدر يشوف الـ Cluster ويتعرف عليه.
- **سياسة التخزين (Conflict Resolution):** ضبطت الخاصية على **replace** أو **append** حسب الحاجة. في حالتنا اخترت **append** أو إني أضيف رقم عشوائي لاسم الملف عشان ما يسمح البيانات القديمة لو تشابهت الأسماء.